

RANSAC: a deep dive

The lab slides and the Lab 3 notebook, explained step by step, with practice questions

What this covers. The lab slides *Robust model fitting by RANSAC* (Tekla Tóth, based on Dániel Baráth's work) and the *Lab 3 notebook* (RANSAC for line fitting). This guide complements the quiz guide on linear systems, distances and RANSAC: it goes deeper into *why* RANSAC works, *how* the notebook implements it, and what goes wrong in practice. Slide numbers are shown as [slide n], and notebook material as [notebook].

The five facts most likely to be asked in the quiz

1. **Minimal sample sizes** (slide 27 lists exactly these): line $m = 2$, plane $m = 3$, sphere $m = 4$, ellipse $m = 5$. Also a 2D circle $m = 3$.
2. **Number of iterations:** $k \geq \frac{\log(1 - \eta)}{\log(1 - \epsilon^m)}$. The slides write η (confidence), ϵ (inlier ratio) and m (sample size); the notebook writes p , w and s for the same things.
3. **The loop:** sample a minimal set, fit a model, count the inliers within a threshold σ , keep the so-far-the-best model, then refit on all its inliers.
4. **RANSAC's per-point cost** is 0 if the distance is within σ and 1 otherwise. So $J(\theta)$ is the number of outliers, and minimizing it means maximizing the number of inliers.
5. **The inlier test uses the point-model distance.** For a normalized line this is $|ax + by + c|$ (or $|\mathbf{x} \cdot \mathbf{n} - r|$ in radial form).

Notation: slides vs. notebook

Quantity	Slides	Notebook	Meaning
confidence	η	confidence (p)	required probability of drawing at least one all-inlier sample
inlier ratio	$\epsilon = Q/N$	ratio (w)	fraction of points that are inliers
sample size	m	sample_size (s)	points needed by the minimal solver
threshold	σ	sigma	maximum distance for a point to count as an inlier
inliers / points	Q / N	inliers / N	$Q = N - J^*$, where J^* is the best number of outliers
model	$\theta, e(S)$	line_params	parameters estimated from sample S

Roadmap. §1 why we need robust fitting, how to parametrize a line, and which loss functions exist · §2 the algorithm and the probability behind “how many samples” · §3 the notebook line by line, with four pitfalls found by re-running it · §4 minimal solvers, parameter tuning, multi-model fitting and RANSAC variants · §5 practice questions · cheat sheet.

1 Foundations: why we need robust fitting

1.1 Motivation [slide 3–7]

Real measurements contain **inliers**, which follow the model up to noise, and **outliers**, which do not follow it at all. The slides give three examples:

- Matching features between a historical photo and today's street view, or finding a book cover in a cluttered image. Many feature matches are wrong, and a transformation (for example a homography) must be estimated from the correct ones only.
- Detecting facade planes, doors and windows in a 3D point cloud.
- Interpreting lidar scans (a traffic sign, a pole, vegetation).

Other applications [slide 7] include detecting geometric primitives (lines and circles, e.g. coins), estimating epipolar geometry, detecting planar surfaces, and segmenting multiple motions.

1.2 Taxonomy of geometric estimation problems [slide 6]

Problem	What is given and sought	Typical tool
SCSI: standard single-class, single-instance	One model (e.g. one line), noisy points, <i>no outliers</i>	Least squares (SVD / eigenvector)
R-SCSI: robust SCSI	One model, noisy points <i>plus outliers</i>	RANSAC (this lab)
SCMI: single-class, multiple-instance	Several models of <i>one</i> type (e.g. three lines) plus outliers	Sequential RANSAC, multi-model methods
MCMI: multi-class, multiple-instance	Several models of <i>different</i> types (lines <i>and</i> circles)	Multi-class, multi-model fitting

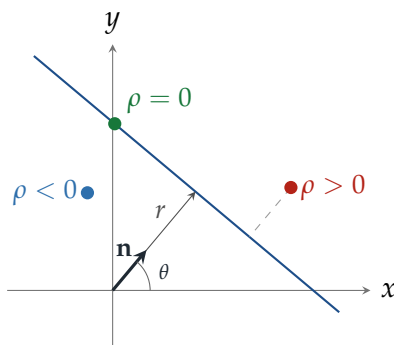
1.3 Line parametrizations and the signed distance [slide 9–10]

Homogeneous (implicit) form: $ax + by + c = 0$ with $(a \neq 0 \vee b \neq 0)$. It has 3 parameters but only 2 degrees of freedom, because (a, b, c) and $\lambda(a, b, c)$ describe the same line.

Radial form (Hesse normal form):

$$x \cos \theta + y \sin \theta = r, \quad \theta \in [0, \pi[, \quad r \in \mathbb{R}, \quad \mathbf{n} = (\cos \theta, \sin \theta), \quad \|\mathbf{n}\| = 1.$$

Here $\mathbf{n}$ is the *unit normal*, and $|r|$ is the distance of the line from the origin. There are exactly 2 parameters and no scale ambiguity.



Signed distance of a point $\mathbf{x} = (x, y)$:

$$\rho(\mathbf{x}) = \mathbf{x} \cdot (\cos \theta, \sin \theta) - r.$$

- $\rho = 0$: the point is on the line.
- $\rho > 0$: the point is on the side the normal points to (away from the origin when $r > 0$).
- $\rho < 0$: the point is on the other side.
- $|\rho(\mathbf{x})|$ is the perpendicular distance: the lecture's point-line formula with $\|\mathbf{n}\| = 1$ built in.

Converting between the two forms. Divide (a, b, c) by $\sqrt{a^2 + b^2}$. This gives $\mathbf{n} = (a, b) / \sqrt{a^2 + b^2}$ and $r = -c / \sqrt{a^2 + b^2}$. If the resulting θ falls outside $[0, \pi[$, flip the signs of both $\mathbf{n}$ and r . The

notebook's `fit_line_two_points` returns exactly this normalized form: $(a, b, c) = (\cos \theta, \sin \theta, -r)$, up to a common sign. That is why its distance function is simply $|ax + by + c|$.

1.4 Model fitting as an optimization problem; loss functions [slide 11–15]

Given points $\mathcal{X} = \{\mathbf{x}_j\}_{j=1}^N$, we look for

$$\theta^* = \arg \min_{\theta} \sum_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}, \theta),$$

where the **per-point cost** f decides everything.

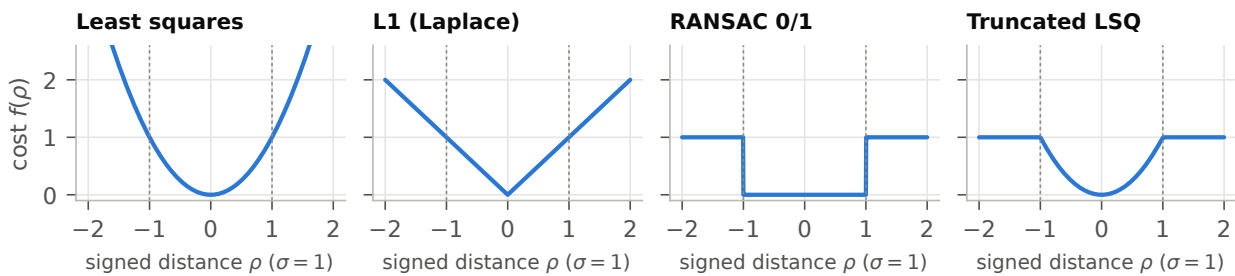


Figure 1. Per-point cost functions of the signed distance ρ , with threshold $\sigma = 1$ (dashed): $f_{LSQ} = \rho^2$; $f_{L1} = |\rho|$; $f_{RANSAC} = 0$ if $|\rho| \leq \sigma$, else a constant; truncated LSQ $= \min(\rho^2, \sigma^2)$, which is the idea behind MSAC.

- **Least squares** $f_{LSQ} = \rho^2$ [slide 12]. This is the *maximum-likelihood* estimate for Gaussian noise: if $\rho \sim \mathcal{N}(0, \sigma^2)$, then $-\log p(\rho) = \rho^2 / (2\sigma^2) + \text{const}$. Gauss used it around 1800. For a line it is solved in closed form by SVD or eigenvectors (centroid plus the smallest eigenvector, as in the lecture).
- **Sum of absolute differences** $f = |\rho|$. Laplace considered it in the late 1700s. It is the ML estimate for Laplacian noise, and it is less sensitive to outliers, but outliers still pull the result.
- **RANSAC** [slide 15, 23]: $f(\mathbf{x}, \theta) = 0$ if the distance is $\leq \sigma$, otherwise a constant (1 in the pseudo-code). Then $J(\theta) = \sum f$ is the **number of outliers**, so minimizing J means **maximizing the consensus set** (the inliers). An outlier costs the same whether it is 2 px or 2000 px away, so a single gross outlier cannot dominate the result.
- **Truncated LSQ** $\min(\rho^2, \sigma^2)$: inliers are scored by how well they fit, and outliers pay a fixed price. This is the idea behind MSAC (§4.4).

Why least squares fails with outliers [slide 13–14]. Squaring gives a point with residual 10σ a hundred times the influence of a point at 1σ . The slides show two failure modes. In Example 1, uniformly scattered outliers tilt the line. In Example 2, a *structured* cluster of outliers drags the line towards itself. In robust-statistics terms, least squares has a *breakdown point* of 0: one arbitrarily bad point can move the estimate arbitrarily far.

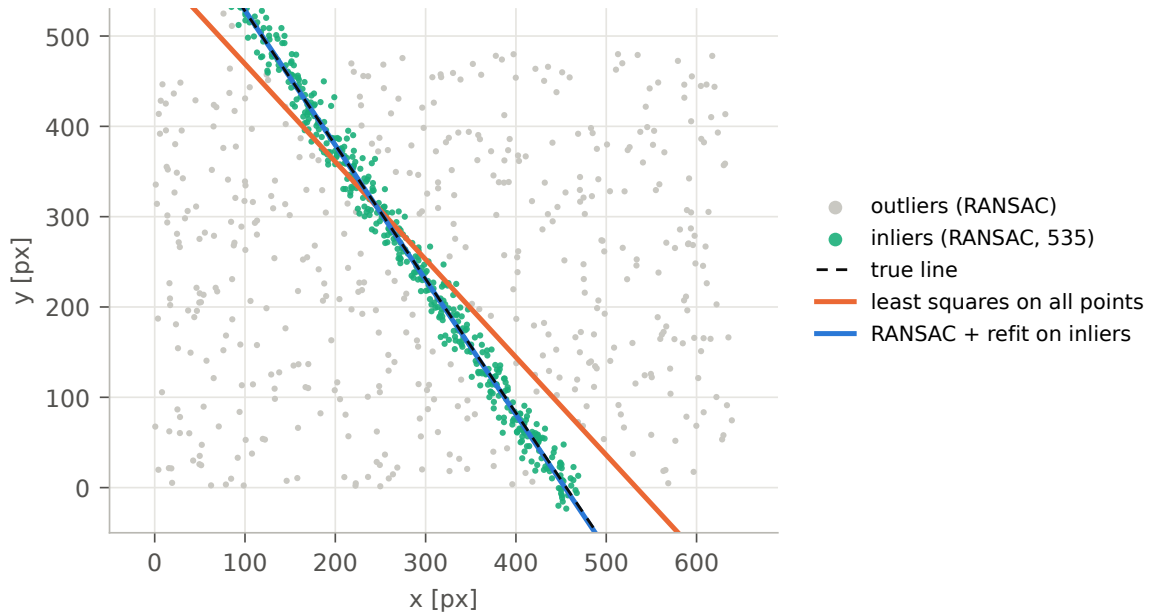


Figure 2. The Lab 3 data (seed 0, $N = 1000$, 50% outliers). Least squares on all points (orange) is off by 8.7° . RANSAC ($\sigma = 25$) followed by a refit on its 535 inliers (blue) is off by 0.15° and lies on top of the true line (dashed).

2 The RANSAC algorithm

2.1 The algorithm of Fischler and Bolles (1981) [slide 16–23]

Algorithm 1 RANSAC, as written on slide 23

Input: data points $\mathcal{X} = \{\mathbf{x}_j\}_{j=1}^N$; a minimal solver $e(S) = \theta$ that estimates model parameters from a sample $S \subseteq \mathcal{X}$; threshold σ ; confidence η .

Cost of one point: $f(\mathbf{x}, \theta) = 0$ if $\text{dist}(\mathbf{x}, \theta) \leq \sigma$, else 1 $\Rightarrow J(\theta) = \sum_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}, \theta)$ is the number of outliers.

Output: θ^* , the model that minimizes J .

```

1:  $iter \leftarrow 0, J^* \leftarrow \infty$ 
2: repeat
3:   Select a random  $S \subseteq \mathcal{X}$  with  $|S| = m$  SAMPLING
4:   Estimate the parameters  $\theta = e(S)$ 
5:   Evaluate  $J(\theta) = \sum_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}, \theta)$  VERIFICATION
6:   if  $J(\theta) < J^*$  then SO-FAR-THE-BEST
7:      $\theta^* \leftarrow \theta, J^* \leftarrow J(\theta)$ 
8:   end if
9:    $iter \leftarrow iter + 1$ 
10: until the probability that a better solution exists, given  $|\mathcal{X}|, J^*$  and  $iter$ , drops below the tolerance set by  $\eta$ 
11: Compute  $\theta^*$  from all inliers  $\mathcal{X}_{in}$ :  $\theta^* \leftarrow \text{LocalOptimization}(\mathcal{X}_{in}, \theta^*)$ 

```

The three phases in words.

- **Sampling.** Draw m distinct points uniformly at random, where m is the *minimal* number the solver needs (2 for a line).
- **Verification.** Compute the residual of *every* point with respect to the hypothesis, and count the points within σ . This is the expensive step: it costs $O(N)$ per iteration.
- **So-far-the-best.** Keep the hypothesis with the largest consensus set (the fewest outliers).

The loop stops once enough samples have been drawn (§2.2). A final *local optimization* then re-estimates

the model from all inliers, typically by least squares.

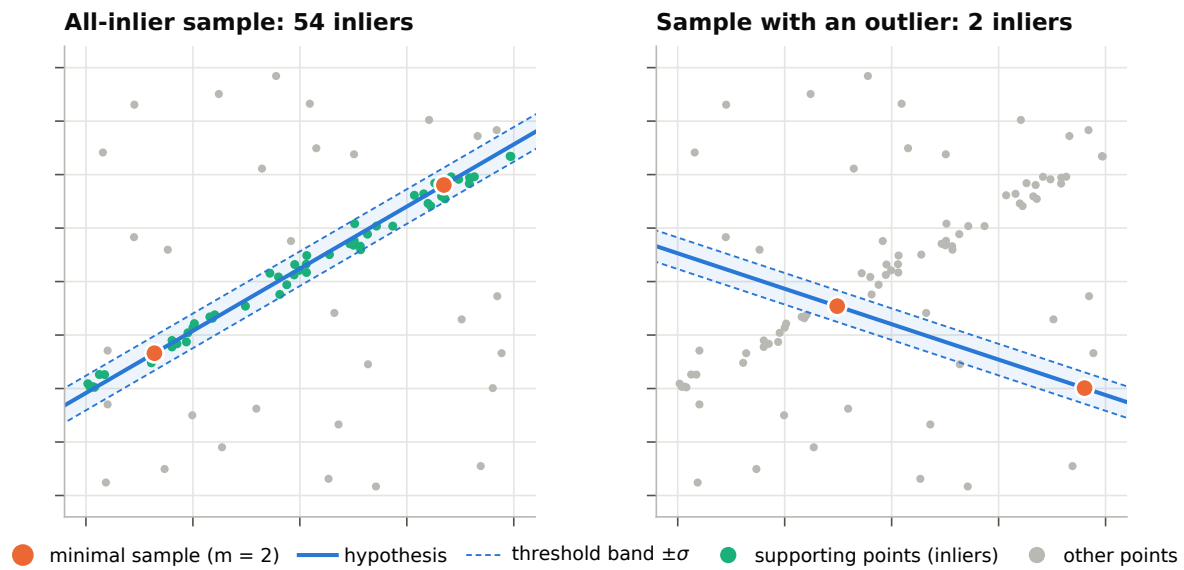


Figure 3. One RANSAC iteration. A minimal sample (orange) defines a hypothesis. Points inside the $\pm\sigma$ band support it. An all-inlier sample with well-separated points gets large support (left). A sample containing an outlier gets almost none (right).

Why minimal samples? Every extra point in the sample multiplies the chance of drawing an outlier-free sample by another factor $\epsilon < 1$. The minimal sample therefore maximizes $P(\text{all-inlier sample})$ and minimizes the number of iterations. Accuracy is recovered afterwards by the refit on all inliers.

2.2 How many samples? The probabilistic guarantee [slide 24–27]

Notation [slide 25]: N is the number of points, $Q = N - J^*$ the number of inliers, m the sample size, and $\epsilon = Q/N$ the inlier ratio.

Probability of an all-inlier (uncontaminated) sample. Sampling m distinct points *without replacement*:

$$P(\text{inlier sample}) = \frac{\binom{Q}{m}}{\binom{N}{m}} \approx \epsilon^m.$$

The approximation is excellent for large N . With $N = 1000$, $Q = 500$, $m = 4$: 0.0621 (exact) against 0.0625. It gets worse for tiny sets. With $N = 20$, $Q = 10$, $m = 4$: 0.0433 against 0.0625.

On average, one sample in $1/P$ is all-inlier [slide 25]. The number of draws until the first good sample follows a geometric distribution with mean $1/P$. For a line with 50% inliers that is $1/0.25 = 4$ draws, and for a sphere $1/0.0625 = 16$.

Deriving the number of iterations. Each sample is contaminated with probability $1 - \epsilon^m$, so all k samples are contaminated with probability $(1 - \epsilon^m)^k$. Requiring this to be at most $1 - \eta$:

$$(1 - \epsilon^m)^k \leq 1 - \eta \implies k \geq \frac{\log(1 - \eta)}{\log(1 - \epsilon^m)} \quad (\text{use } \lceil \cdot \rceil; \text{ the log base cancels})$$

Small- ϵ^m approximation: $\log(1 - x) \approx -x$ gives $k \approx \frac{-\ln(1 - \eta)}{\epsilon^m}$, which is about $3/\epsilon^m$ for $\eta = 0.95$ and $4.6/\epsilon^m$ for $\eta = 0.99$. So k grows like ϵ^{-m} : **polynomially in $1/\epsilon$** (as slide 30 says) and **exponentially in m** .

$\eta = 0.95$	$\epsilon = 15\%$	20%	30%	40%	50%	70%
$m = 2$ (line)	132	73	32	17	10	4
$m = 3$ (plane, circle)	886	373	109	45	22	7
$m = 4$ (sphere; homography later)	5916	1871	368	116	46	11
$m = 5$ (ellipse)	39 448	9360	1231	291	94	16
$m = 7$ (fundamental matrix, 7-point)	$1.75 \cdot 10^6$	$2.34 \cdot 10^5$	$1.37 \cdot 10^4$	1827	382	35
$m = 8$ (fundamental matrix, 8-point)	$1.17 \cdot 10^7$	$1.17 \cdot 10^6$	$4.57 \cdot 10^4$	4570	765	50

The rows $m = 2, 4, 7, 8$ are the slide's table [slide 26–27]. The rows $m = 3, 5$ are computed the same way. **Note:** the slide rounds to the *nearest* integer; the strict bound uses the ceiling, e.g. $\lceil 10.4 \rceil = 11$ for $m = 2, \epsilon = 50\%$. The “ ∞ ” entries on the slide (e.g. $m = 30, \epsilon = 15\%$) are not truly infinite: $1 - \epsilon^m$ rounds to exactly 1 in floating point, so $\log(1 - \epsilon^m) = 0$. The true values are astronomically large.

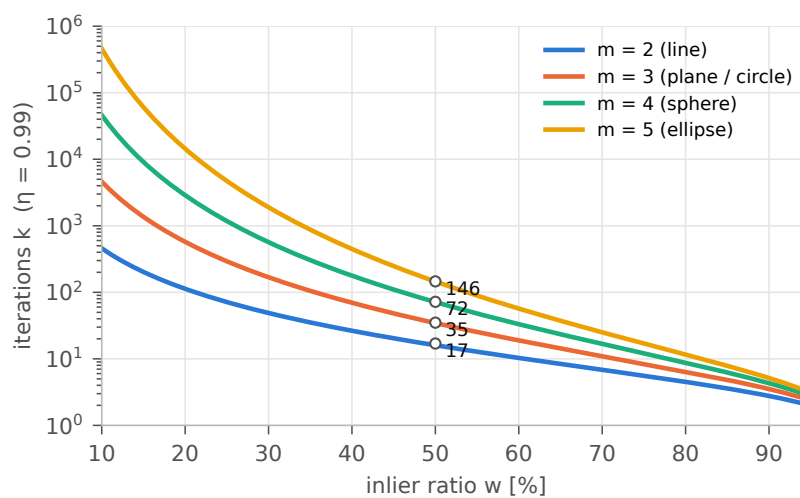


Figure 4. Required iterations for $\eta = 0.99$ (log scale). At 50% inliers: 17 (line), 35 (plane/circle), 72 (sphere), 146 (ellipse). Each extra point in the minimal sample multiplies k by roughly $1/\epsilon$.

2.3 Adaptive termination

The inlier ratio ϵ is *unknown* in advance. Adaptive RANSAC starts from a maximum number of iterations. Whenever it finds a new so-far-the-best model, it re-estimates $\hat{\epsilon} = Q/N$ and recomputes k . Since $\hat{\epsilon}$ is at most the true ratio, k is conservative: it can only overestimate. Here is a trace of a correctly written adaptive loop on the Lab 3 data ($\eta = 0.95, m = 2, \sigma = 25$):

iteration	best inliers Q	$\hat{\epsilon}$	new bound k
1	84	0.084	424
2	244	0.244	49
3	249	0.249	47
14	535	0.535	9 $\Rightarrow$ stop after iteration 14

2.4 Pros and cons [slide 28]

Pros	Cons
Extremely popular: the 1981 paper has tens of thousands of citations.	Slow if the inlier ratio is low, since $k \sim \epsilon^{-m}$.
Used in many applications: primitives, homographies, epipolar geometry, point clouds.	Observed to take <i>several times longer</i> than the theory predicts, because of noise. . .
The percentage of inliers is not needed in advance and is not limited (it works even below 50%).	. . . and because not every all-inlier sample generates a good hypothesis: $P(\text{inlier sample}) \neq P(\text{good model estimate})$.
A probabilistic guarantee for the solution.	The threshold σ has to be known or chosen.
Mild assumptions: only σ must be known.	The result is random, and the minimal-sample model is noisy (hence the refit).

Why can an all-inlier sample still be bad? Two inliers that lie close together determine the *direction* of the line poorly, because their noise is large relative to their separation. The band then misses many true inliers, the consensus stays small, and RANSAC keeps searching.

3 The Lab 3 notebook, line by line

3.1 Synthetic data: `generate_data(N, p, direction, inlier_ratio)`

- The image is 640×480 . The direction $\mathbf{d}$ is normalized. There are $N_{in} = \lfloor N \cdot \text{ratio} \rfloor$ inliers and $N_{out} = N - N_{in}$ outliers.
- Inliers:** $\mathbf{q} = \mathbf{p} + t \mathbf{d}$ with $t \sim U(-\frac{\text{diag}}{2}, \frac{\text{diag}}{2})$ and $\text{diag} = \sqrt{640^2 + 480^2} = 800$. Gaussian noise $\mathcal{N}(0, 10^2)$ is then added to *each* coordinate. Because the noise is isotropic, the *perpendicular* offset of an inlier from the true line is also $\mathcal{N}(0, 10^2)$, so its standard deviation is 10 px.
- Outliers:** uniform in $[0, 640] \times [0, 480]$. Inliers can fall outside the image (the segment is 800 px long), which is why the plots show the range $-50 \dots 690$.
- The run uses $N = 1000$, ratio 0.5, $\mathbf{p} = (240, 320)$, and a random direction angle (seed 0).

3.2 The minimal solver: `fit_line_two_points`

```
def fit_line_two_points(p1, p2):
    x1, y1 = p1
    x2, y2 = p2
    dx, dy = x2 - x1, y2 - y1
    norm = math.sqrt(dx**2 + dy**2)
    if norm < 1e-8:          # identical points -> degenerate sample
        return None
    a, b = -dy / norm, dx / norm    # unit normal (-dy, dx)/|d|
    c = -a * x1 - b * y1           # line passes through p1
    return a, b, c
```

The direction is $(\Delta x, \Delta y)$, so $(-\Delta y, \Delta x)$ is perpendicular to it. Dividing by the length makes $a^2 + b^2 = 1$, and c makes the line pass through $\mathbf{p}_1$. **Worked example:** for $(1, 1)$ and $(4, 5)$, $\Delta = (3, 4)$ and $\|\Delta\| = 5$, so $(a, b) = (-0.8, 0.6)$ and $c = 0.2$. Check with the second point: $-3.2 + 3 + 0.2 = 0$.

3.3 The residual: `distance_point_line`

```
def distance_point_line(line_params, point):
    a, b, c = line_params
    x, y = point
    return abs(a*x + b*y + c)    # valid because a^2 + b^2 = 1
```

This is $|\rho(\mathbf{x})|$, the general formula $|ax + by + c|/\sqrt{a^2 + b^2}$ with a denominator equal to 1.

3.4 Adaptive iteration count: `iteration_number`

```
def iteration_number(confidence, sample_size, inliers, N):
    if inliers == 0:
        return float('inf')
    ratio = inliers / float(N)
    top = math.log(1.0 - confidence)
    bottom = math.log(1.0 - ratio**sample_size)
    return math.ceil(top / bottom) if bottom != 0 else float('inf')
```

This is $k = \lceil \log(1 - p) / \log(1 - w^s) \rceil$. The guards cover $Q = 0$ and w^s so small that $1 - w^s$ rounds to 1. **Edge case:** if every point is an inlier ($w = 1$), then `math.log(0)` raises `ValueError`; a production version should return 1.

3.5 The main loop: `ransac_line`

```
def ransac_line(points, max_iterations=2000, sigma=10.0, confidence=0.95):
    best_inliers_count = 0
    best_line, best_inliers_list = None, []
    for _ in range(max_iterations):
        idx1, idx2 = np.random.choice(len(points), 2, replace=False) # SAMPLING
        line_params = fit_line_two_points(points[idx1], points[idx2])
        if line_params is None:
            continue
        inliers = [pt for pt in points # VERIFICATION
                  if distance_point_line(line_params, pt) < sigma]
        if len(inliers) > best_inliers_count: # SO-FAR-THE-BEST
            best_inliers_count, best_line, best_inliers_list = len(inliers), line_params, inliers
            max_iterations = min(max_iterations,
                                iteration_number(confidence, 2, best_inliers_count, len(points)))
    return best_line, np.array(best_inliers_list)
```

3.6 Four pitfalls found by re-running the notebook

1. The adaptive stop never takes effect. In Python, `range(max_iterations)` is evaluated *once*, when the loop starts. Reassigning `max_iterations` inside the loop changes the printed number but **not** the loop. Re-running the notebook code (seed 0), the bound fell to 23 at the 9th iteration, yet all 2000 iterations were executed. The result is still correct (the best model is kept), just wasteful. **Fix:** use `while it < k:` and recompute `k` inside the loop (§3.7). On this data the corrected loop stops after 9–34 iterations (on average 27 with $t = 10$ and 12 with $t = 25$).

2. The threshold equals one noise standard deviation. The perpendicular noise has $\sigma_n = 10$ px, and `sigma=10` keeps only about 68% of the true inliers (measured: 337 of 500). A common rule is a threshold of about $2\text{--}3\sigma_n$; for a point–line distance (1 degree of freedom), $t = \sqrt{3.84}\sigma_n \approx 1.96\sigma_n$ keeps 95%.

threshold t	10 ($1\sigma_n$)	19.6 ($1.96\sigma_n$)	25 ($2.5\sigma_n$)	30 ($3\sigma_n$)
true inliers inside the band (of 500)	337	478	497	500
outliers inside the band (of 500)	13	35	42	54

This is a trade-off: a larger t keeps more true inliers but also accepts more outliers. The refit (pitfall 3) limits the damage.

3. The refinement step is announced but not implemented. The notebook's plan says "Refine the best line using all inliers", but `ransac_line` returns the *two-point* model. Refitting with total least squares (the centroid plus the smallest eigenvector, from the lecture) clearly helps. Averaged over 20 runs:

	$t = 10$	$t = 25$
angle error, minimal-sample model	0.59°	1.48°
angle error, after refit on inliers	0.46°	0.33°
iterations executed (correct adaptive loop)	26.8	12.1

```
def refit_line(inlier_pts):
    m = inlier_pts.mean(axis=0)
    X = inlier_pts - m
    w, V = np.linalg.eigh(X.T @ X)
    n = V[:, 0]
    return n[0], n[1], -n @ m
```

total least squares
optimal line passes through the centroid
eigenvalues in ascending order
normal = smallest eigenvector
(a, b, c) with $a^2 + b^2 = 1$

4. The saved output does not match a fresh run. The notebook shows the iteration bounds 256, 125, 10, 10 and "517 inliers". Running the code exactly as written (seed 0, noise 10) gives a different trace and about 362 inliers, which is consistent with pitfall 2 (68% of 500, plus a few outliers). The stored output was probably produced by an earlier state of the notebook. Do not memorize those numbers. Note also that the line (a, b, c) and $(-a, -b, -c)$ are the same line, so the sign of the result can flip between runs.

Performance note. The list comprehension loops over the points in Python. Vectorizing the check as `d = np.abs(P @ np.array([a, b]) + c)` and `mask = d < sigma` does the same work in one NumPy call.

3.7 A corrected, generic implementation (tested)

The same loop works for any model once you provide a minimal solver, a residual function and a refit. The code below was run on synthetic data. For a sphere (300 inliers with noise 0.02, plus 300 outliers, $\sigma = 0.06$, $\eta = 0.99$), it recovered the center (0.998, 2.008, 3.001) and $R = 1.999$ (true values (1, 2, 3), $R = 2$) in 75 iterations, close to the theoretical 72.

```
def num_iterations(eta, w, m):
    if w <= 0: return math.inf
    if w >= 1: return 1
    return math.ceil(math.log(1 - eta) / math.log(1 - w ** m))

def ransac(X, m, fit_minimal, residuals, refit, sigma, eta=0.99, max_iter=10_000, rng=None):
    rng = rng or np.random.default_rng()
    best_model, best_mask, best_q = None, None, 0
    k, it = max_iter, 0
    while it < k:
        it += 1
        S = X[rng.choice(len(X), m, replace=False)] # SAMPLING
        model = fit_minimal(S) # minimal solver
        if model is None: # degenerate sample
            continue
        mask = residuals(X, model) < sigma # VERIFICATION (vectorized)
        q = int(mask.sum())
        if q > best_q: # SO-FAR-THE-BEST
            best_model, best_mask, best_q = model, mask, q
            k = min(k, num_iterations(eta, q / len(X), m)) # adaptive k
    if best_model is None:
        return None, None, it
    return refit(X[best_mask]), best_mask, it # LOCAL OPTIMIZATION (refit)
```

4 Beyond the line: solvers, parameters, variants

4.1 Minimal solvers for common primitives

For a minimal sample, the lecture's fitting systems become *square*, so they are solved exactly. The refit uses the same equations, but overdetermined and solved by least squares.

Model	m	Minimal solver	Residual for the inlier test	Degenerate if
2D line	2	$\mathbf{n} = (-\Delta y, \Delta x) / \ \Delta\ , c = -\mathbf{n} \cdot \mathbf{p}_1$	$ \mathbf{n} \cdot \mathbf{x} + c $	points coincide
3D line	2	direction $\mathbf{v} = (\mathbf{p}_2 - \mathbf{p}_1) / \ \cdot\ $	$\ (\mathbf{x} - \mathbf{p}_1) - ((\mathbf{x} - \mathbf{p}_1) \cdot \mathbf{v})\mathbf{v}\ $	points coincide
Plane	3	$\mathbf{n} = (\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_3 - \mathbf{p}_1)$, normalized; $d = -\mathbf{n} \cdot \mathbf{p}_1$	$ \mathbf{n} \cdot \mathbf{x} + d $	collinear
2D circle	3	solve the 3×3 system with rows $[1 \ -2x_i \ -2y_i]$ and rhs $-x_i^2 - y_i^2$	$ \ \mathbf{x} - \mathbf{c}\ - R $	collinear (det = 0)
Sphere	4	solve the 4×4 system with rows $[1 \ -2x_i \ -2y_i \ -2z_i]$	$ \ \mathbf{x} - \mathbf{c}\ - R $	coplanar
Ellipse	5	null vector of the 5×6 matrix $[x^2 \ xy \ y^2 \ x \ y \ 1]$ (last row of $\mathbf{V}^T$); keep it only if $B^2 - 4AC < 0$	algebraic residual $ [x^2 \ xy \ y^2 \ x \ y \ 1] \mathbf{z} $, or a geometric approximation	4 collinear

```
def plane_from_3(p1, p2, p3, eps=1e-8):
    n = np.cross(p2 - p1, p3 - p1); nrm = np.linalg.norm(n)
    if nrm < eps: return None # collinear -> degenerate
    n = n / nrm
    return n, -n @ p1 # n.x + d = 0

def circle_from_3(P, eps=1e-8):
    # P: 3x2
    A = np.c_[np.ones(3), -2 * P] # rows [1, -2x, -2y]
    b = -(P ** 2).sum(1) # -x^2 - y^2
    if abs(np.linalg.det(A)) < eps: return None # collinear -> degenerate
    k, x0, y0 = np.linalg.solve(A, b) # k = x0^2 + y0^2 - R^2
    c = np.array([x0, y0]); return c, np.sqrt(c @ c - k)

def sphere_from_4(P, eps=1e-8):
    # P: 4x3, same idea in 3D
    A = np.c_[np.ones(4), -2 * P]; b = -(P ** 2).sum(1)
    if abs(np.linalg.det(A)) < eps: return None # coplanar -> degenerate
    k, *c = np.linalg.solve(A, b); c = np.array(c)
    return c, np.sqrt(c @ c - k)

def conic_from_5(P):
    # homogeneous 5x6 system
    x, y = P[:, 0], P[:, 1]
    M = np.c_[x**2, x*y, y**2, x, y, np.ones(5)]
    z = np.linalg.svd(M)[2][-1] # null vector of M
    A, B, C = z[:3]
    return z if B**2 - 4*A*C < 0 else None # keep ellipses only
```

All four solvers were tested on synthetic data: they recover the true plane, circle, sphere and ellipse to machine precision, and they return None for collinear or coplanar samples.

4.2 Parameter settings and their effects

Parameter	If you increase it...	How to choose it
threshold σ	more true inliers are accepted, but so are more outliers; hypotheses look better, so k decreases	about 2–3 times the noise standard deviation of the residual (e.g. $1.96\sigma_n$ keeps 95% for a 1-DoF distance)
confidence η	more iterations ($k \propto -\ln(1 - \eta)$): 0.95 gives about 3, 0.99 about 4.6, and 0.999 about 6.9, each divided by ϵ^m	0.95 in the notebook; 0.99 is common
sample size m	k grows like ϵ^{-m}	always the <i>minimal</i> number for the model
max iterations	a hard cap on run time when ϵ is very low	a safety limit (2000 in the notebook)
number of points N	k is unchanged, but each verification costs more ($O(N)$)	—

4.3 Multiple instances (SCMI) with sequential RANSAC

To find several models of the same class, such as three lines or two facade planes:

1. Run RANSAC on the remaining points and take the best model.
2. If its consensus is smaller than a minimum size, stop.
3. Otherwise, remove its inliers, store the model, and go back to step 1.

Caveats. The method is greedy: a wrong early model can “steal” points from two true structures (e.g. a line through the crossing of two lines), and points near an intersection can be assigned to the wrong model. For MCMI, several model classes compete for the points, which calls for dedicated multi-class, multi-model methods.

4.4 RANSAC upgrades [slide 29–30]

Vanilla RANSAC is not “case closed”. Every highlighted part of the pseudo-code (the random sampling, the 0/1 cost, the threshold, the minimal-sample estimate, and the so-far-the-best rule) has been improved:

Weak point	Methods	Idea in one line
Cost function	MSAC, MLESAC, Huber loss	Score inliers by how well they fit, not just 0/1. MSAC uses the truncated quadratic $\min(\rho^2, \sigma^2)$; MLESAC maximizes a likelihood with an inlier/outlier mixture model.
Threshold σ	LMedS, MINPRAN, MAGSAC	LMedS minimizes the <i>median</i> squared residual (no threshold needed, but it breaks down beyond 50% outliers). MINPRAN chooses the threshold by statistics. MAGSAC marginalizes over a range of σ values.
Degeneracy	DegenSAC	Detect samples that satisfy the model for the wrong reason (e.g. a dominant plane when estimating epipolar geometry) and handle them separately.
Accuracy	LO-RANSAC, Graph-Cut RANSAC	Run a local optimization (an iterated least-squares re-fit) whenever a new so-far-the-best model appears. GC-RANSAC also uses the spatial coherence of inliers.
Speed	RANSAC with SPRT (WaldSAC), PROSAC	SPRT stops verifying a bad hypothesis early, after a few points fail. PROSAC samples the most promising points first, e.g. the best feature matches.

Speed has two components [slide 30]: the time per iteration grows with the number of points N , and the number

of iterations grows polynomially in $1/\epsilon$ (as ϵ^{-m}).

5 Practice questions: the lab part

The tags are the same as in the quiz guide. The **LAB DETAIL** tag marks questions about notebook details, which are less likely in the quiz but useful for the assignments and the oral exam.

Minimal samples and the number of iterations

L1 Slide 27 in one question

VERY HIGH

Which minimal sample sizes are correct for (line, plane, sphere, ellipse)?

✓ 2, 3, 4, 5 **CORRECT**

- ☐ 2, 3, 3, 5
- ☐ 2, 4, 4, 6
- ☐ 1, 3, 4, 6

Why: Slide 27 lists exactly these values. A plane has 4 coefficients up to scale (3 DoF), a sphere 4 parameters, and a conic 6 coefficients up to scale (5 DoF).

L2 Sphere

VERY HIGH

How many points are required to estimate a sphere minimally in RANSAC?

- ☐ 3
- ✓ 4 **CORRECT**
- ☐ 5
- ☐ 6

Why: The unknowns are x_0, y_0, z_0 and R , so 4 points that are not coplanar are needed.

L3 Iterations (slide notation)

HIGH

The number of samples k needed to hit at least one all-inlier sample with probability η (ϵ : inlier ratio, m : sample size) is

- ☐ $k \geq \log(\eta) / \log(\epsilon^m)$
- ✓ $k \geq \log(1 - \eta) / \log(1 - \epsilon^m)$ **CORRECT**
- ☐ $k \geq \log(1 - \epsilon^m) / \log(1 - \eta)$
- ☐ $k \geq \log(1 - \eta) / \log(1 - \epsilon)^m$

Why: This is the same formula as the notebook's $\log(1 - p) / \log(1 - w^s)$, written in the slides' symbols.

L4 All-inlier probability (exact)

HIGH

What is the probability that a random sample of m distinct points is all-inlier (Q inliers among N points)?

- ✓ $\binom{Q}{m} / \binom{N}{m} \approx \epsilon^m$ **CORRECT**
- ☐ $\binom{N}{m} / \binom{Q}{m}$
- ☐ $1 - \epsilon^m$
- ☐ $m \epsilon$

Why: Count the m -subsets of inliers and divide by all m -subsets. $1 - \epsilon^m$ is the probability that the sample is contaminated.

L5 Average number of draws

MEDIUM

On average, how many samples are drawn until the first all-inlier sample, if P is the probability of an all-inlier sample?

- ☐ P
- ✓ $1/P$ **CORRECT**
- ☐ $\log(1 - \eta)$
- ☐ N/m

Why: The number of draws follows a geometric distribution with mean $1/P$. For a line with 50% inliers that is $1/0.25 = 4$.

L6 Reading the slide table

MEDIUM

From the slide table ($\eta = 0.95$), how many samples are needed for a line ($m = 2$) at 50% inliers?

- ☐ 4
- ✓ 10 **CORRECT**
- ☐ 17
- ☐ 46

Why: $\log 0.05 / \log 0.75 \approx 10.4$. The slide rounds this to 10; the strict ceiling is 11. 17 is the value at 40%, and 46 is for $m = 4$.

L7 Ceiling vs. rounding**NEW FORMAT**

The notebook's `iteration_number(0.95, 2, 500, 1000)` returns

- ☐ 10
- ✓ **11 CORRECT**
- ☐ 17
- ☐ 4

Why: It uses `math.ceil`, so $\lceil 10.41 \rceil = 11$, one more than the slide's rounded value.

L8 Exact vs. approximate**NEW FORMAT**

With $N = 20$, $Q = 10$ and $m = 2$, what is the exact probability of an all-inlier sample?

- ☐ 0.25
- ✓ **$45/190 \approx 0.237$ CORRECT**
- ☐ 0.5
- ☐ $10/190$

Why: $\binom{10}{2} / \binom{20}{2} = 45/190$. The approximation $\epsilon^m = 0.25$ is slightly optimistic for small N .

Cost functions, parametrization, taxonomy**L9 RANSAC's per-point cost****MEDIUM**

In the RANSAC pseudo-code, what is the cost $f(\mathbf{x}, \theta)$ of one data point?

- ☐ $\rho(\mathbf{x})^2$
- ✓ **0 if its distance to the model is $\leq \sigma$, otherwise 1 CORRECT**
- ☐ $|\rho(\mathbf{x})|$
- ☐ $\min(\rho^2, \sigma^2)$

Why: A 0/1 cost. The truncated quadratic $\min(\rho^2, \sigma^2)$ is MSAC.

L11 Least squares and noise**MEDIUM**

Least-squares fitting is the maximum-likelihood method for which type of noise?

- ☐ uniform
- ✓ **normal (Gaussian) CORRECT**
- ☐ Laplacian
- ☐ Poisson

Why: $-\log$ of a Gaussian gives $\rho^2 / (2\sigma^2)$. Laplacian noise leads to $|\rho|$ (Laplace's sum of absolute differences).

L10 Meaning of $J(\theta)$ **MEDIUM**

With the 0/1 cost, $J(\theta) = \sum_{\mathbf{x}} f(\mathbf{x}, \theta)$ equals

- ✓ **the number of outliers CORRECT**
- ☐ the number of inliers
- ☐ the sum of squared distances
- ☐ the median residual

Why: Minimizing J is the same as maximizing the consensus set, $Q = N - J^*$.

L12 Radial line parametrization**MEDIUM**

Which is the radial parametrization of a 2D line used in the slides?

- ☐ $y = mx + b$
- ✓ **$x \cos \theta + y \sin \theta = r$, with $\theta \in [0, \pi[$ and $r \in \mathbb{R}$ CORRECT**
- ☐ $x \sin \theta + y \cos \theta = r$, with $\theta \in [0, 2\pi[$ and $r \geq 0$
- ☐ $ax + by + c = 0$

Why: $\mathbf{n} = (\cos \theta, \sin \theta)$ is the unit normal, and r is the signed distance of the line from the origin.

L13 Signed distance**MEDIUM**

What is the signed distance of $\mathbf{x}$ from the line (θ, r) ?

- ✓ $\rho(\mathbf{x}) = \mathbf{x} \cdot (\cos \theta, \sin \theta) - r$ **CORRECT**
- $\rho(\mathbf{x}) = \|\mathbf{x}\| - r$
- $\rho(\mathbf{x}) = (\mathbf{x} \cdot (\cos \theta, \sin \theta) - r)^2$
- $\rho(\mathbf{x}) = \mathbf{x} \cdot (\sin \theta, \cos \theta) + r$

Why: Its sign tells the side of the line, and it is 0 on the line. Because $\|\mathbf{n}\| = 1$, no division is needed.

L14 Taxonomy**MEDIUM**

Finding *three lines* in a point set that also contains outliers is a

- SCSI problem
- R-SCSI problem
- ✓ **SCMI problem** **CORRECT**
- MCMI problem

Why: It has a single class (lines) and multiple instances. Lines *and* circles together would be MCMI.

L15 The main assumption**MEDIUM**

Which is the (mild) assumption of RANSAC according to the slides?

- the inlier ratio is known
- ✓ **the threshold σ is known** **CORRECT**
- there are fewer than 50% outliers
- the noise is Laplacian

Why: The percentage of inliers is not needed and is not limited. Only σ must be known.

L16 Slower than the theory**MEDIUM**

Why does RANSAC in practice often take several times longer than the theory predicts?

- because the formula for k is wrong
- ✓ **because of noise: not every all-inlier sample gives a good hypothesis** **CORRECT**
- because sampling is done with replacement
- because the threshold is too large

Why: $P(\text{inlier sample}) \neq P(\text{good model estimate})$. Two close-together inliers determine the line direction poorly.

The algorithm and its upgrades**L17 Phases of the loop****MEDIUM**

Which three phases does one RANSAC iteration consist of (slide 23)?

- ✓ **sampling, verification, so-far-the-best update** **CORRECT**
- initialization, gradient step, convergence test
- sampling, least-squares fit on all points, rejection
- clustering, voting, non-maximum suppression

Why: Sample S , estimate $\theta = e(S)$, evaluate $J(\theta)$, keep the model if $J(\theta) < J^*$.

L18 The last line of the pseudo-code**MEDIUM**

What happens after the RANSAC loop in the slides' pseudo-code?

- the best minimal sample is returned
- ✓ **θ^* is re-estimated from all inliers (local optimization)** **CORRECT**
- the threshold is re-estimated
- outliers are refit with a second model

Why: $\theta^* \leftarrow \text{LocalOptimization}(\mathcal{X}_{in}, \theta^*)$, typically a least-squares refit.

L19 Degeneracy**NEW FORMAT**

Which RANSAC variant is designed to handle *degenerate* configurations?

- PROSAC
- MSAC
- ✓ **DegenSAC** **CORRECT**
- LMedS

Why: Slide 30: correctness of the results and degeneracy are solved by DegenSAC.

L20 Accuracy**NEW FORMAT**

Which variants improve *accuracy*, given that parameters are estimated from minimal samples?

- ✓ **LO-RANSAC and Graph-Cut RANSAC** **CORRECT**
- PROSAC and WaldSAC
- MINPRAN and MAGSAC
- MSAC and MLESAC

Why: They run a local optimization on the so-far-the-best model. PROSAC and WaldSAC target speed, MINPRAN and MAGSAC the threshold, MSAC and MLESAC the cost function.

L21 Speed**NEW FORMAT**

Which pair of methods addresses RANSAC's *running time*?

- DegenSAC and LMedS
- ✓ **RANSAC with SPRT (WaldSAC) and PROSAC** **CORRECT**
- MSAC and Huber loss
- LO-RANSAC and MAGSAC

Why: SPRT rejects bad hypotheses early during verification. PROSAC samples the most promising points first.

L22 Threshold-free**NEW FORMAT**

Which method avoids a fixed inlier threshold by minimizing the median of the squared residuals?

- MSAC
- ✓ **Least Median of Squares (LMedS)** **CORRECT**
- PROSAC
- DegenSAC

Why: LMedS needs no σ , but it breaks down beyond 50% outliers. It is listed on slide 30 under "outlier threshold sigma".

Notebook details**L23 Normal from two points****LAB DETAIL**

In `fit_line_two_points`, the coefficients (a, b) are

- $(\Delta x, \Delta y) / \|\Delta\|$
- ✓ **$(-\Delta y, \Delta x) / \|\Delta\|$** **CORRECT**
- $(\Delta y, \Delta x)$
- (x_1, y_1)

Why: This is the direction rotated by 90° and then normalized, so $a^2 + b^2 = 1$. Then $c = -(ax_1 + by_1)$.

L24 No division needed**LAB DETAIL**

Why does `distance_point_line` return $|ax + by + c|$ without dividing by $\sqrt{a^2 + b^2}$?

- because the result is only a rough estimate
- ✓ **because the line is normalized, $a^2 + b^2 = 1$** **CORRECT**
- because $c = 0$
- because only the sign matters

Why: The general formula is $|ax + by + c| / \sqrt{a^2 + b^2}$, and here the denominator is 1.

L25 Notebook parameters**LAB DETAIL**

Which confidence and threshold does `ransac_line` use by default?

- ☐ 0.99 and 3
- ✓ ☒ 0.95 and 10 **CORRECT**
- ☐ 0.95 and 1
- ☐ 0.5 and 10

Why: `max_iterations=2000, sigma=10.0, confidence=0.95`.

L26 The loop pitfall**LAB DETAIL**

Inside `for _ in range(max_iterations):`, the code reassigns `max_iterations`. What is the effect?

- ☐ the loop stops as soon as the bound is reached
- ✓ ☒ none: `range` was created once, so all 2000 iterations run **CORRECT**
- ☐ Python raises an error
- ☐ the loop restarts

Why: The adaptive bound is computed but never enforced. A `while` loop fixes this.

L27 Threshold vs. noise**NEW FORMAT**

The inliers have perpendicular noise with standard deviation 10 px, and the threshold is 10 px. Roughly what fraction of the true inliers falls inside the band?

- ☐ 50%
- ✓ ☒ 68% **CORRECT**
- ☐ 95%
- ☐ 99.7%

Why: $P(|\rho| < 1\sigma) \approx 68\%$ (measured: 337 of 500). Keeping 95% needs a threshold of about 1.96σ .

L28 A bad all-inlier sample**NEW FORMAT**

Why can a sample of two *inliers* still produce a poor line hypothesis?

- ☐ because inliers have no noise
- ✓ ☒ because if the two points are close together, the noise dominates their separation and the direction is inaccurate **CORRECT**
- ☐ because two points never define a line
- ☐ because the threshold is applied before sampling

Why: This is one reason why RANSAC runs longer than predicted, and why a refit on all inliers is needed.

L29 True or false (lab)**NEW FORMAT**

Statement	Answer
(a) RANSAC needs the percentage of inliers in advance.	False
(b) With the 0/1 cost, an outlier 2 px beyond the threshold costs the same as one 2000 px away.	True
(c) Least squares has a breakdown point of 0: a single gross outlier can ruin it.	True
(d) The number of iterations grows exponentially with the sample size m .	True
(e) The number of iterations depends directly on the total number of points N .	False
(f) The “ ∞ ” entries in the slide table mean that RANSAC can never succeed.	False
(g) (a, b, c) and $(-a, -b, -c)$ describe the same line.	True
(h) The radial form $x \cos \theta + y \sin \theta = r$ has a scale ambiguity, like $ax + by + c = 0$.	False
(i) The approximation $P \approx \epsilon^m$ ignores that sampling is without replacement.	True

Why: (a) Only σ must be known. (e) N enters only through ϵ , but the time per iteration grows with N . (f) The entries are floating-point overflow; the true k is finite but huge. (h) The radial form has exactly 2 parameters and a unit normal.

RANSAC cheat sheet

The algorithm [slide 23]

Repeat: sample m random points $\rightarrow$ estimate $\theta = e(S)$ with the minimal solver $\rightarrow$ verify: $J(\theta) = \#\{x : \text{dist}(x, \theta) > \sigma\} \rightarrow$ if $J < J^*$, keep it as so-far-the-best $\rightarrow$ update k .

Finally: LocalOptimization, i.e. refit by least squares on all inliers (a line or plane goes through the centroid, with the smallest eigenvector as normal).

Per-point cost: 0 if $\leq \sigma$, otherwise 1. Least squares uses ρ^2 (ML for Gaussian noise, Gauss), L1 uses $|\rho|$ (Laplace), MSAC uses $\min(\rho^2, \sigma^2)$.

How many samples [slide 25–27]

$P(\text{all-inlier}) = \binom{Q}{m} / \binom{N}{m} \approx \epsilon^m$, on average one sample in $1/P$ is all-inlier.

$k \geq \frac{\log(1-\eta)}{\log(1-\epsilon^m)} \approx \frac{-\ln(1-\eta)}{\epsilon^m}$ (about $3/\epsilon^m$ for $\eta = 0.95$, $4.6/\epsilon^m$ for $\eta = 0.99$). Independent of N , exponential in m .

At 50% inliers: $\eta = 0.95$ gives 10 / 22 / 46 / 94 (slide rounding) for $m = 2/3/4/5$; $\eta = 0.99$ gives 17 / 35 / 72 / 146.

Notation: slides use η, ϵ, m ; the notebook uses confidence, ratio, sample_size (p, w, s).

Minimal samples

2D line	plane	2D circle	sphere	ellipse	3D line	parabola	3D circle
2	3	3	4	5	2	3	3

Degenerate samples: coincident (line), collinear (plane, circle), coplanar (sphere). Reject them and draw again.

Line geometry [slide 9–10]

Homogeneous form: $ax + by + c = 0$ (3 parameters, 2 DoF). Radial form: $x \cos \theta + y \sin \theta = r$, with $\theta \in [0, \pi[$ and $r \in \mathbb{R}$.

Signed distance: $\rho(x) = x \cdot (\cos \theta, \sin \theta) - r$. For a normalized (a, b, c) from two points, $(a, b) = (-\Delta y, \Delta x) / \|\Delta\|$, $c = -(ax_1 + by_1)$, and $\text{dist} = |ax + by + c|$.

Taxonomy, pros/cons, upgrades [slide 6, 28, 30]

SCSI (least squares) · R-SCSI (RANSAC) · SCMI (sequential RANSAC) · MCMI.

Pros: popular, inlier ratio not needed, probabilistic guarantee, only σ assumed. **Cons:** slow for low ϵ , slower than the theory predicts (noise; an all-inlier sample is not always a good model).

Upgrades: cost function: MSAC, MLESAC, Huber · threshold: LMedS, MINPRAN, MAGSAC · degeneracy: DegenSAC · accuracy: LO-RANSAC, GC-RANSAC · speed: SPRT (WaldSAC), PROSAC.

Notebook pitfalls: (1) for `_ in range(max_iterations)` ignores the updated bound, so all 2000 iterations run; use `while`. (2) A threshold of 10 equals $1\sigma_n$ and keeps only about 68% of the inliers; use $2-3\sigma_n$. (3) The “refine on inliers” step is missing; add a total least-squares refit. (4) The stored output (517 inliers) does not match a fresh run.